LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 8 ACARA 19

SEMESTER 3



Tugas :
Postman dan API authentication

Disusun Oleh:
Uaryl Engno Laurenca (E41250317)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PENDAHULUAN

Pada acara sebelumnya, API Sistem Informasi Akademik sudah memiliki endpoint CRUD dengan arsitektur MVC (Acara 17) dan format response yang seragam lewat class `Response` (Acara 18). Namun API tersebut masih terbuka, artinya siapa pun dapat mengaksesnya tanpa identitas. Agar API dapat membedakan pengguna dan nantinya membatasi akses, diperlukan sistem autentikasi.

Acara 19 mempersiapkan fondasi autentikasi dengan membuat tabel `users`, Model `User`, `AuthController`, dan endpoint `POST /register` yang menyimpan kata sandi dalam bentuk hash. Aksi login yang menghasilkan token ditambahkan pada Acara 20. Pada acara ini Postman juga diposisikan sebagai alat pengujian utama: request dikelompokkan dalam Collection, alamat server diatur lewat Environment, dan contoh response disimpan sebagai dokumentasi awal.

B. TUJUAN PRAKTIKUM

1. Memahami konsep pengujian API (API Testing) dan komponen HTTP request serta response pada Postman.
2. Memahami skema autentikasi pada REST API: Basic Authentication, API Key, dan Bearer Token (JWT).
3. Membuat Collection dan Environment pada Postman dengan variabel base_url.
4. Merancang tabel users dan menyimpan kata sandi dengan password_hash().
5. Membuat Model User, AuthController, dan pemetaan resource register pada Router.
6. Menguji endpoint register dan mendokumentasikan contoh response sukses dan gagal.
7. Menambahkan validasi email dengan filter_var() dan membuat Environment kedua untuk simulasi staging.

C. DASAR TEORI

3.1 Pengujian API (API Testing)

API Testing adalah proses menguji API untuk memastikan setiap endpoint berfungsi sesuai spesifikasi. Berbeda dengan pengujian antarmuka pengguna, API Testing berfokus pada komunikasi client dan server: validitas request, struktur response, keakuratan data, performa, dan penanganan kesalahan. Aspek yang diperhatikan meliputi validitas endpoint, metode HTTP, parameter request, struktur JSON, waktu respons, kode status HTTP, serta mekanisme autentikasi dan otorisasi.

3.2 Postman

Postman adalah aplikasi untuk mengembangkan, menguji, mendokumentasikan, dan memantau API dengan antarmuka grafis. Fitur utamanya: mengirim HTTP request, mengatur header, mengirim parameter URL, mengirim Body (JSON atau Form Data), menyimpan Collection, otomatisasi pengujian sederhana, dan dokumentasi API.

3.3 Komponen HTTP Request dan Response

- Method: GET (ambil data), POST (tambah data), PUT (ubah seluruh data), PATCH (ubah sebagian data), DELETE (hapus data).
- URL endpoint: alamat resource, contoh {{base_url}}/mahasiswa.
- Header: informasi tambahan, contoh Content-Type: application/json, Accept: application/json, dan Authorization: Bearer xxxxx.
- Body: data yang dikirim ke server, terutama untuk POST dan PUT.
- Response terdiri dari status code, response header, response body, response time, dan response size.

3.4 Autentikasi pada REST API

- Basic Authentication: username dan password di-encode Base64 dan dikirim pada setiap request.
- API Key: token unik untuk klien, dikirim lewat header atau parameter.
- Bearer Token (termasuk JWT): token yang diperoleh setelah login berhasil dan disertakan pada header Authorization di setiap request berikutnya.

Untuk aplikasi dengan satu jenis klien yang dikendalikan sendiri oleh pengembang, token sederhana yang stateless seperti Bearer Token umumnya sudah memadai dan lebih ringan dibanding OAuth yang dirancang untuk banyak aplikasi pihak ketiga.

3.5 Penyimpanan Kata Sandi

Kata sandi tidak boleh disimpan sebagai teks biasa. Menurut dokumentasi PHP Manual, password_hash() dengan PASSWORD_DEFAULT direkomendasikan untuk membuat hash yang aman, dan password_verify() dipakai untuk mencocokkan kata sandi yang dimasukkan pengguna saat login dengan hash yang tersimpan.

D. HASIL DAN PEMBAHASAN

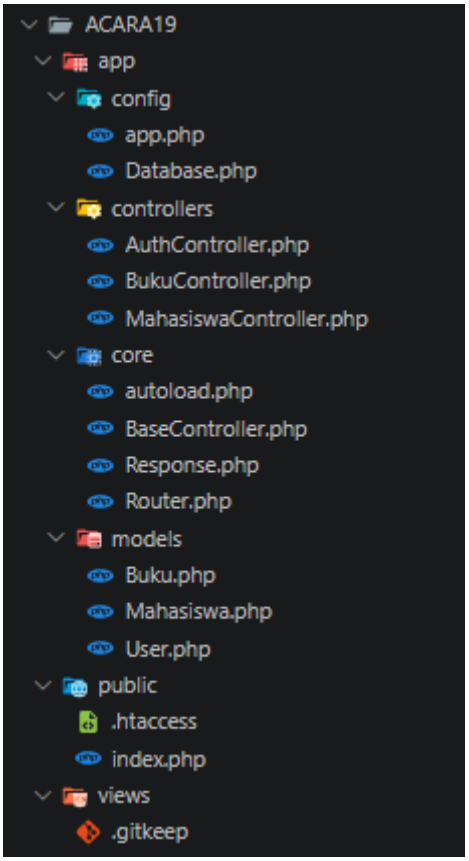
4.1 Alat dan Bahan

XAMPP (Apache, PHP, MySQL), Postman, Visual Studio Code, Git dan GitHub, dan web browser.

4.2 Gambaran Perubahan

Acara ini menambahkan satu resource baru ke arsitektur MVC yang sudah berjalan:

- db_workshop.sql : ditambah tabel users
- app/models/User.php : baru, Model pengguna
- app/controllers/AuthController.php : baru, aksi register()
- app/core/Router.php : resource register dipetakan ke AuthController
- Collection dan dua Environment Postman : baru
    - Model Mahasiswa, Buku, class Response, dan Controller lainnya tidak diubah.

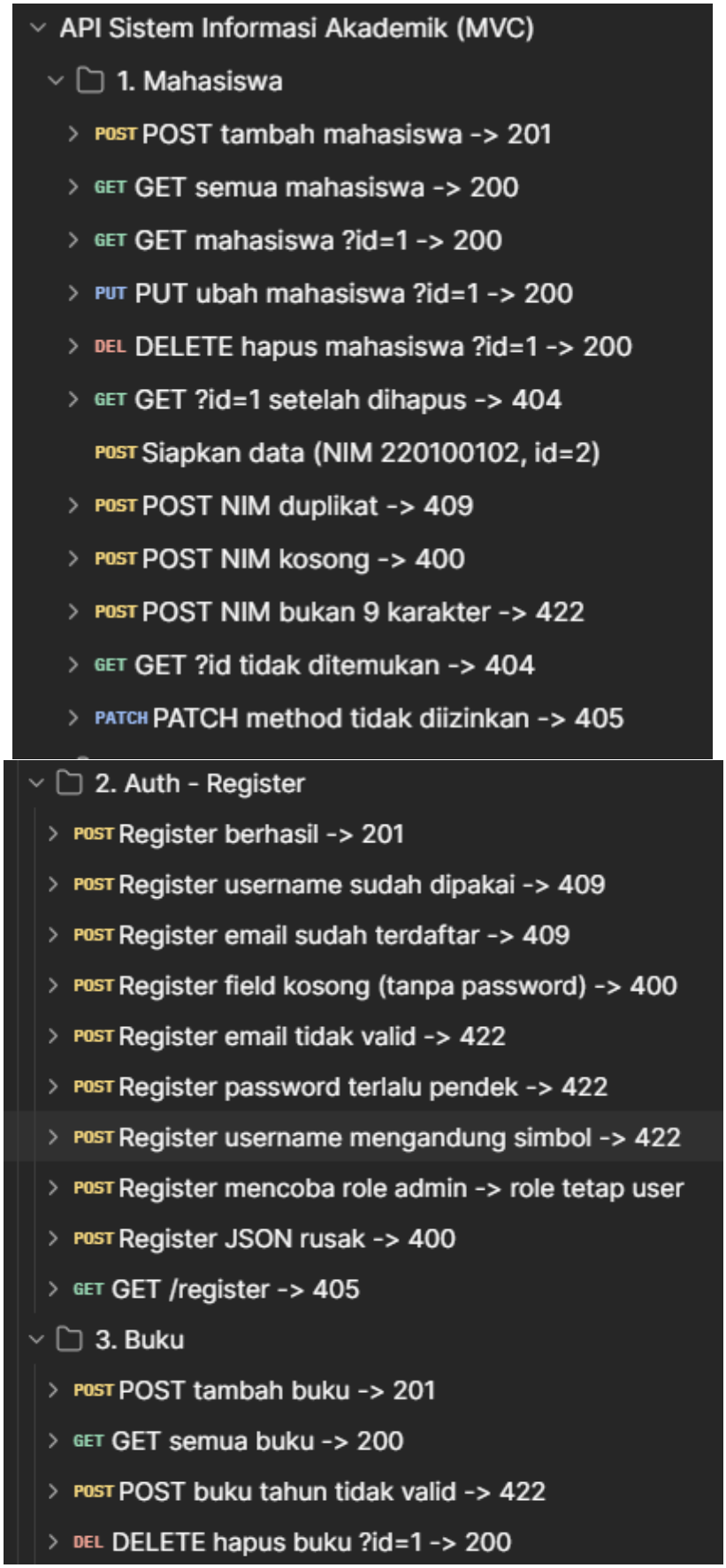


[SCREENSHOT 1: struktur folder proyek di VS Code, terlihat User.php dan AuthController.php]

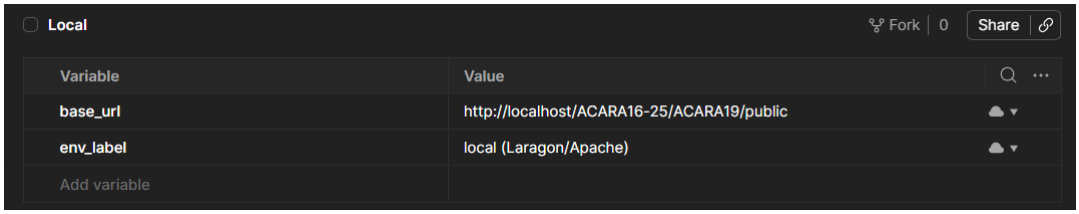
4.3 Langkah 1: Collection dan Environment Postman

1. Dibuat Collection bernama “API Sistem Informasi Akademik (MVC)”.
2. Dibuat Environment “Local” dengan variabel base_url bernilai http://localhost/sistem-informasi-akademik/public. Endpoint tidak memakai akhiran .php karena request diarahkan oleh Front Controller dan Router.
3. Request mahasiswa dari Acara 17 dan 18 dipindahkan ke Collection ini dengan alamat {{base_url}}/mahasiswa.

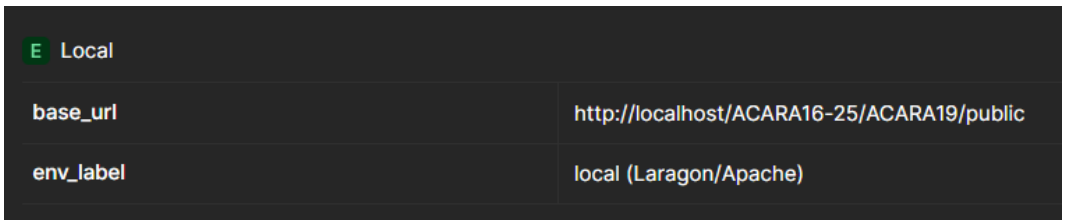
Collection dibagi menjadi tiga folder: 1. Mahasiswa, 2. Auth - Register, dan 3. Buku.



[SCREENSHOT 2: Collection di Postman, terlihat tiga folder dan daftar request]



[SCREENSHOT 3: Environment "Local" dengan variabel base_url]



[SCREENSHOT 4: environment Local terpilih di pojok kanan atas Postman]

4.4 Langkah 2: Tabel users

Tabel users dibuat dengan kolom id (primary key), username (unik), email (unik), password (VARCHAR 255 untuk menyimpan hash), role (enum admin atau user, default user), dan created_at. Kolom password dibuat 255 karakter agar aman jika algoritma PASSWORD_DEFAULT berubah di versi PHP berikutnya.

[SCREENSHOT 5: struktur tabel users di phpMyAdmin]

4.5 Langkah 3: Model User

User.php berisi seluruh akses database untuk tabel users dengan prepared statement:

- create() menyimpan pengguna baru (kata sandi sudah berupa hash).
- findByUsername() dan findByEmail() mencari pengguna.
- validate() memeriksa format username, email, dan password.
- isValidEmail() memeriksa email dengan filter_var() (tugas mandiri 1).



[SCREENSHOT 6: kode User.php]

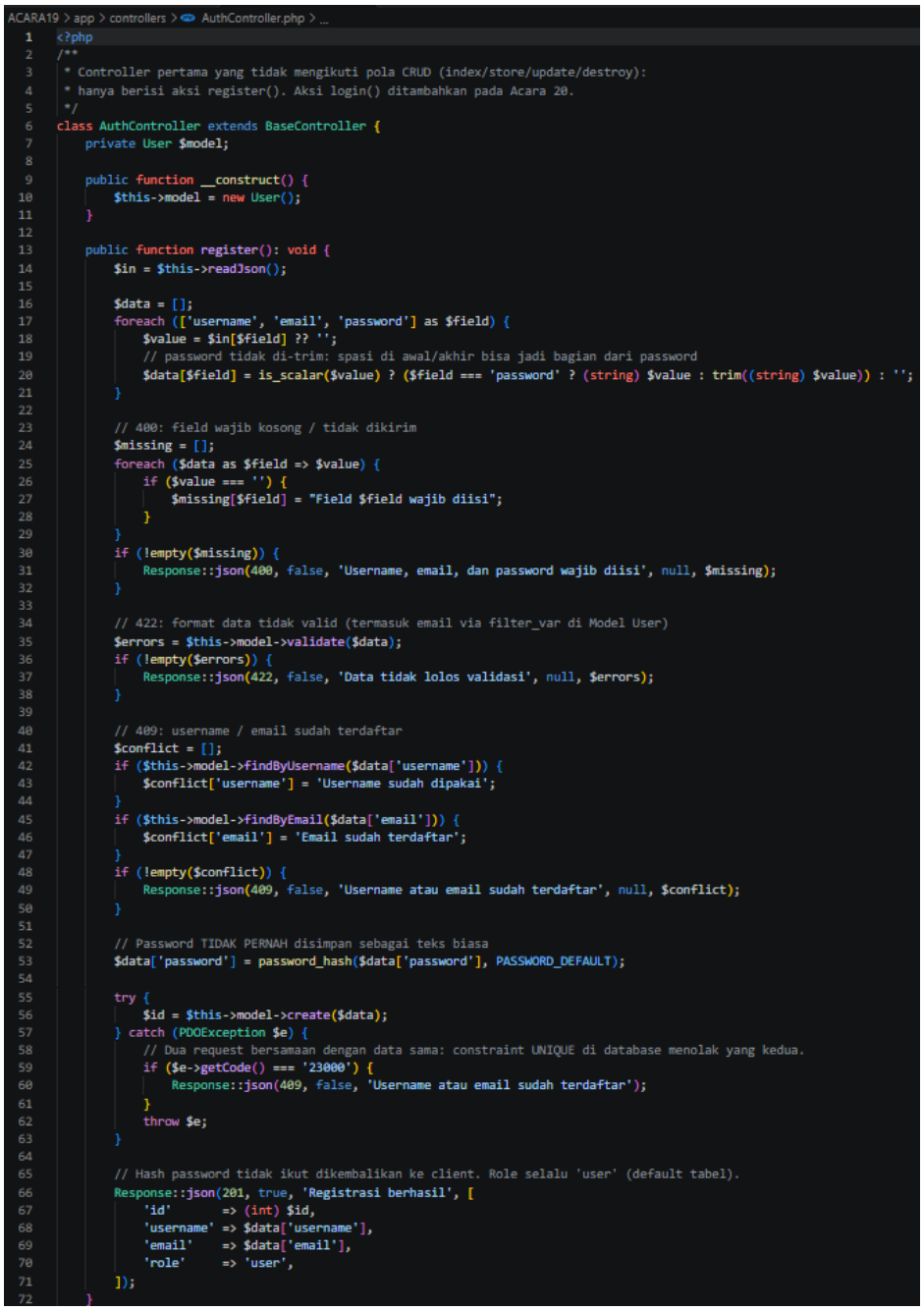
4.6 Langkah 4: AuthController

AuthController adalah Controller pertama yang tidak mengikuti pola CRUD standar (index, store, update, destroy) karena hanya berisi satu aksi, yaitu register(). Aksi login() ditambahkan pada Acara 20. Urutan pemeriksaan pada register():

1. 400 Bad Request jika username, email, atau password kosong.
2. 422 Unprocessable Entity jika format data tidak valid (email salah, password kurang dari 8 karakter, username mengandung simbol).
3. 409 Conflict jika username atau email sudah terdaftar.

4. Kata sandi di-hash dengan password_hash($password, PASSWORD_DEFAULT), lalu data disimpan.
5. 201 Created dengan data pengguna (id, username, email, role). Hash kata sandi tidak ikut dikirim ke client.

Kolom role tidak pernah diambil dari input client, sehingga pendaftar tidak bisa menjadikan dirinya admin. Constraint UNIQUE pada tabel menjadi pengaman terakhir jika dua request dengan data sama masuk bersamaan.



[SCREENSHOT 7: kode AuthController.php]

4.7 Langkah 5: Mendaftarkan Resource register pada Router
Karena register bukan resource CRUD biasa, Router diberi penanganan khusus sebelum blok switch method HTTP. Peta resource ditambah 'register' => 'AuthController'. Jika resource adalah register, Router langsung memanggil register() dan menolak method selain POST dengan status 405 dan header Allow: POST.

ACARA19 > app > core > Router.php > ...

```php
<?php
class Router {
    // Peta sederhana: nama resource pada URL => nama class Controller
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',
        'register'  => 'AuthController',   // Acara 19
    ];

    public function dispatch(): void {
        $uri = trim(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH), '/');
        $segments = $uri === '' ? [] : explode('/', $uri);

        // Cari nama resource dari ujung URL, sehingga tetap jalan di folder
        // mana pun proyek ditaruh (mis. /sistem-informasi-akademik/public/mahasiswa).
        $resource = null;
        $controllerName = null;
        $last = end($segments);
        if ($last !== false && isset($this->map[$last])) {
            $resource = $last;
            $controllerName = $this->map[$last];
        }

        if (!$controllerName) {
            Response::json(404, false, 'Endpoint tidak ditemukan');
        }

        $controller = new $controllerName();

        // 'register' bukan resource CRUD biasa: hanya aksi register() lewat POST.
        if ($resource === 'register') {
            if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
                header('Allow: POST');
                Response::json(405, false, 'Method tidak diizinkan, gunakan POST');
            }
            $controller->register();
            return;
        }

        switch ($_SERVER['REQUEST_METHOD']) {
            case 'GET':    $controller->index(); break;
            case 'POST':   $controller->store(); break;
            case 'PUT':    $controller->update($_GET['id'] ?? null); break;
            case 'DELETE': $controller->destroy($_GET['id'] ?? null); break;
            default:
                header('Allow: GET, POST, PUT, DELETE');
                Response::json(405, false, 'Method tidak diizinkan');
        }
    }
}

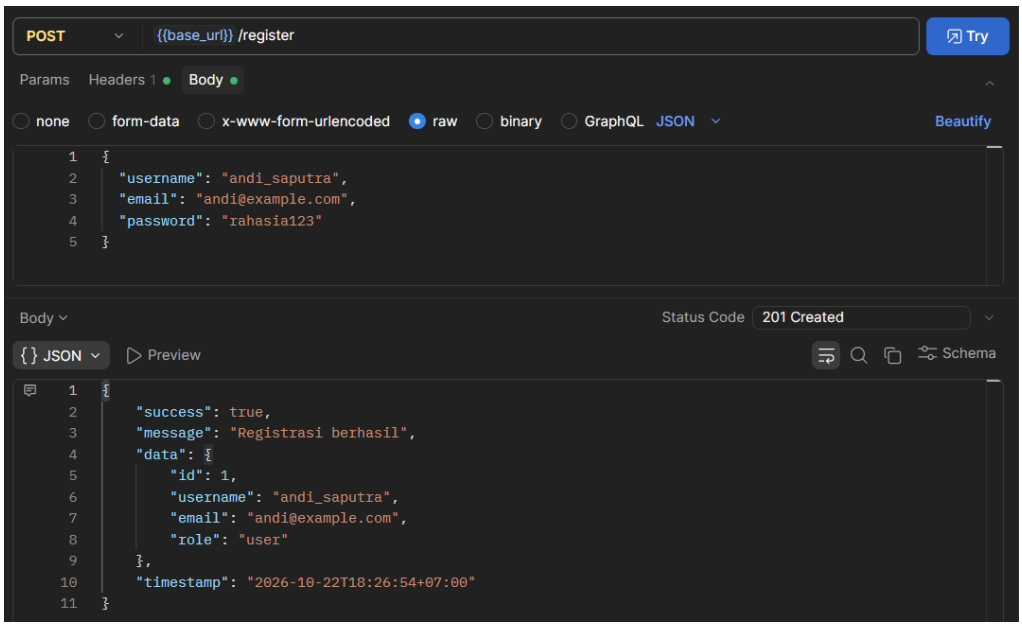
```

[SCREENSHOT 8: kode Router.php, terlihat peta resource dan penanganan register]

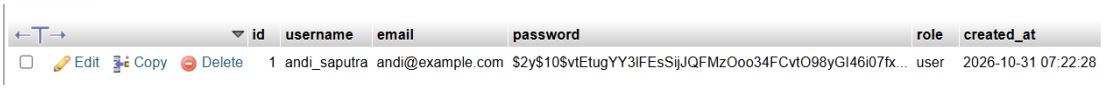
4.8 Langkah 6: Pengujian dan Dokumentasi di Postman
Request POST {{base_url}}/register ditambahkan ke Collection dan diuji dengan beberapa data percobaan. Setiap request memiliki test otomatis untuk status code,

header Content-Type, response time di bawah 1000 ms, serta kelengkapan format response (success, message, data, timestamp).

(1) Registrasi berhasil
Body:
{“username”:”andi_saputra”,”email”:”andi@example.com”,”password”:”rahasia123"
}
Hasil: status 201 Created, message “Registrasi berhasil”, data berisi id, username, email, dan role “user”. Tidak ada field password pada response.

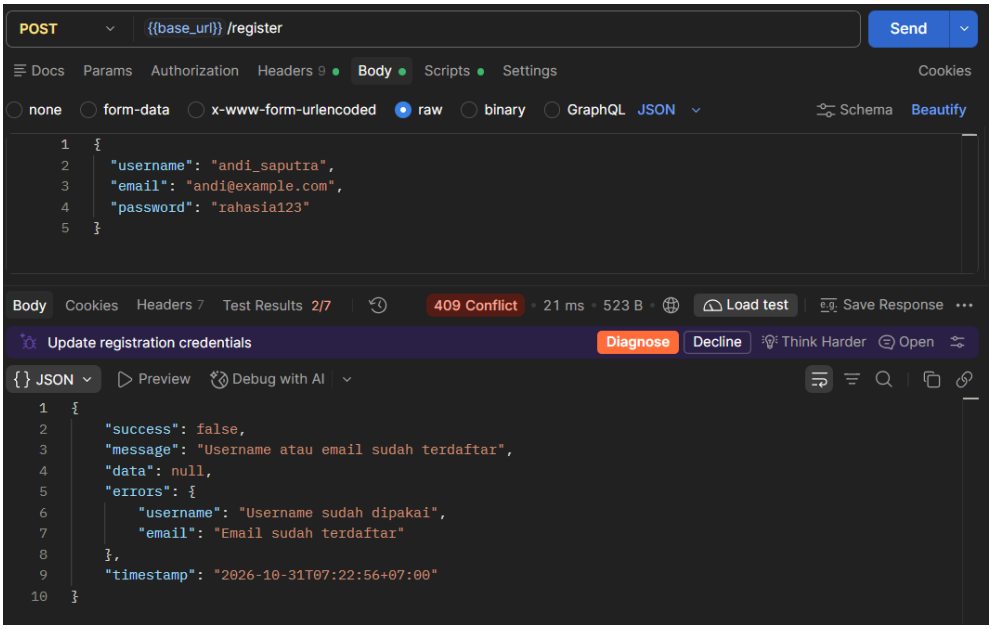


[SCREENSHOT 9: Postman POST /register, body request dan response 201]



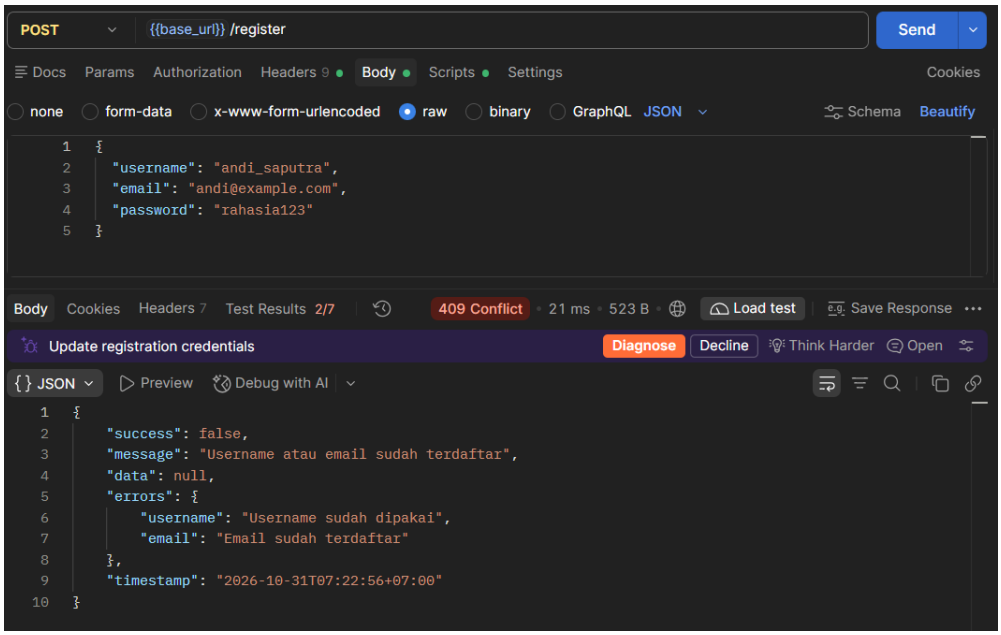
[SCREENSHOT 10: tabel users di phpMyAdmin, kolom password berisi hash berawalan $2y$ (60 karakter), bukan teks asli]

(2) Username sudah dipakai
Hasil: status 409 Conflict, errors berisi {“username”: “Username sudah dipakai”}.



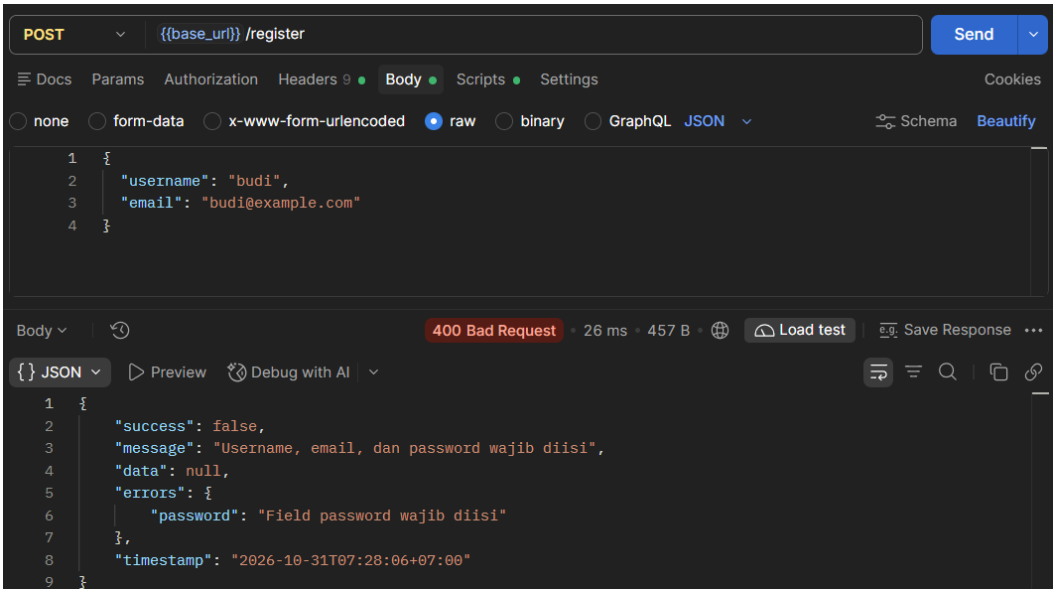
[SCREENSHOT 11: Postman register username duplikat, status 409]

(3) Email sudah terdaftar
Hasil: status 409 Conflict, errors berisi {“email”: “Email sudah terdaftar”}.



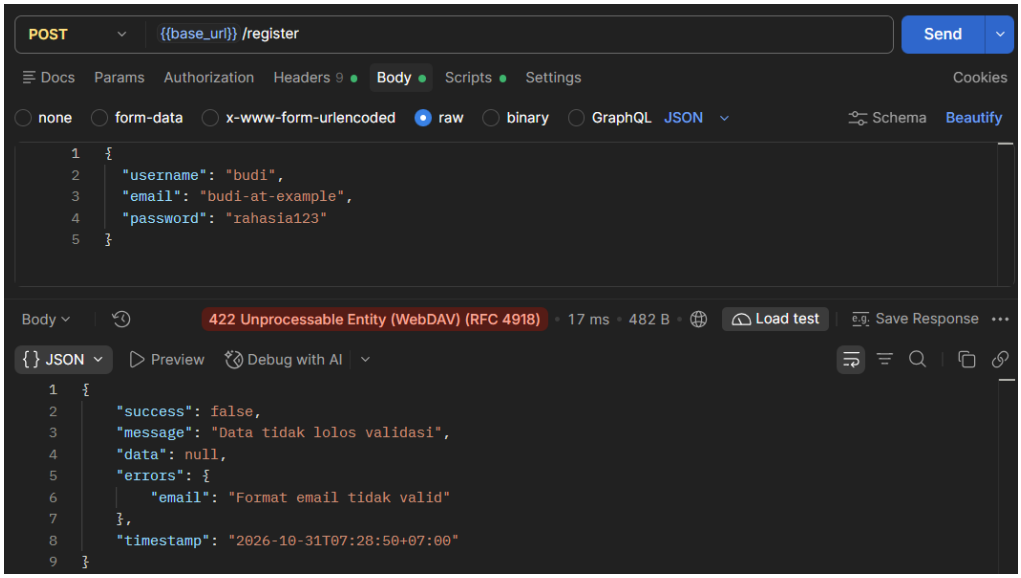
[SCREENSHOT 12: Postman register email duplikat, status 409]

(4) Field kosong
Body tanpa password. Hasil: status 400 Bad Request, errors berisi {“password”: “Field password wajib diisi”}.



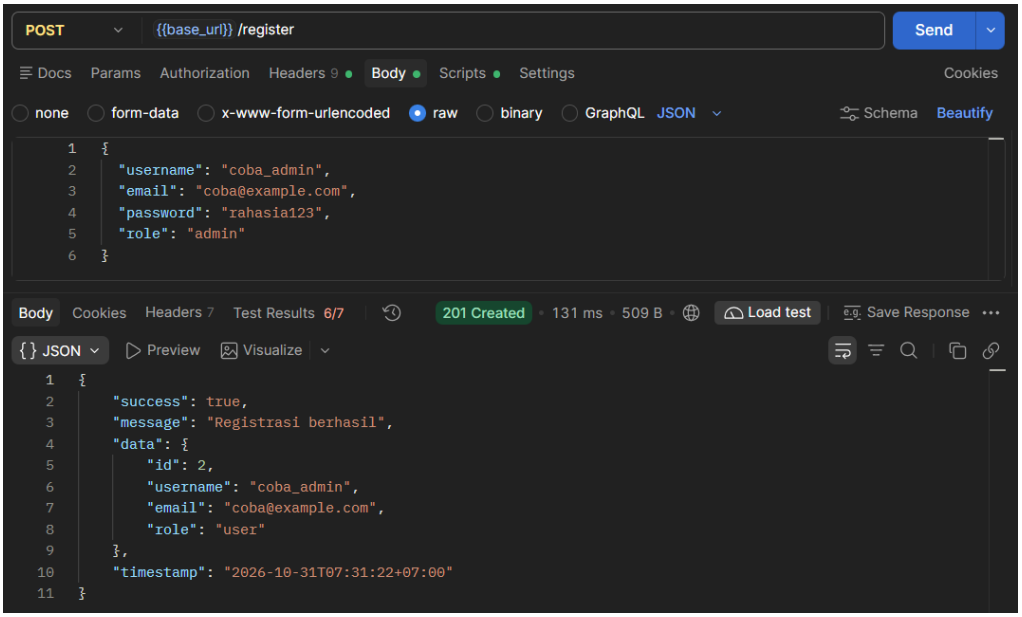
[SCREENSHOT 13: Postman register tanpa password, status 400]

(5) Email tidak valid
Body dengan email “budi-at-example”. Hasil: status 422, errors berisi {“email”: “Format email tidak valid”}.



[SCREENSHOT 14: Postman register email tidak valid, status 422]

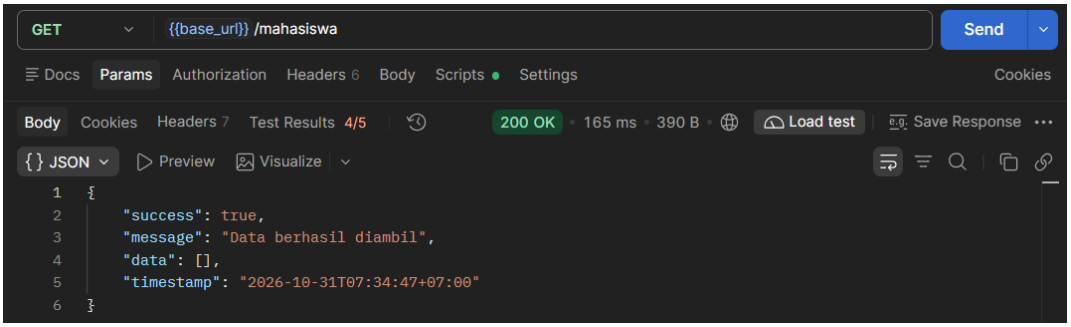
(6) Pengujian tambahan
- Password kurang dari 8 karakter: status 422.
- Username mengandung simbol: status 422.
- Body menyertakan “role”:”admin”: registrasi tetap berhasil (201) tetapi role pada response dan database tetap “user”.
- JSON rusak: status 400.
- Method GET ke /register: status 405.



[SCREENSHOT 15: Postman register dengan "role":"admin", response menunjukkan role "user"]

(7) Contoh response tersimpan (Saved Example)
Contoh response sukses dan gagal disimpan pada masing-masing request sebagai dokumentasi awal, sehingga tampil di bawah nama request di sidebar Collection.

(8) Pengujian ulang request mahasiswa dengan base_url baru
Request GET /mahasiswa dikirim ulang dari Collection baru untuk memastikan endpoint lama tetap berfungsi.

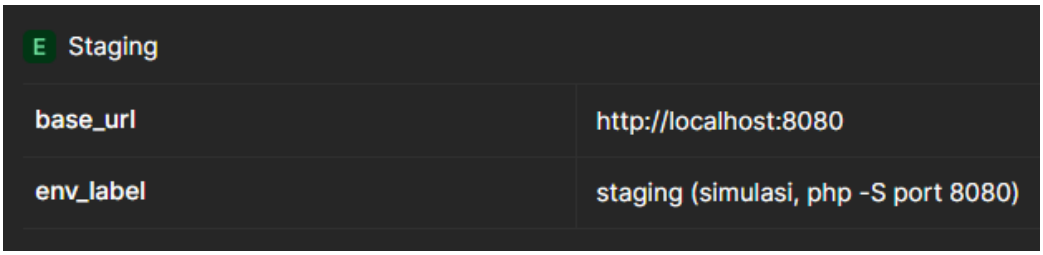


[SCREENSHOT 17: Postman GET {{base_url}}/mahasiswa, status 200]

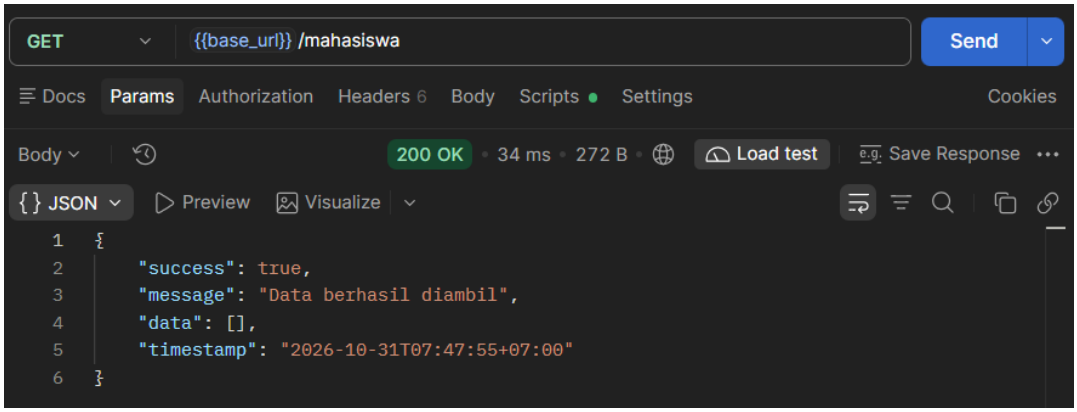
4.9 Tugas Mandiri 1: Validasi Email pada Model User
Fungsi User::isValidEmail() memakai filter_var($email, FILTER_VALIDATE_EMAIL). Validasi dilakukan dua kali: pertama oleh User::validate() yang dipanggil Controller sehingga email salah dibalas 422 beserta pesan per field, kedua di dalam User::create() sebagai pengaman terakhir sebelum data disimpan (melempar InvalidArgumentException bila email tidak valid). Pengujiannya sudah ditunjukkan pada *SCREENSHOT 14.*

4.10 Tugas Mandiri 2: Environment Staging

Dibuat Environment kedua bernama “Staging” dengan base_url = http://localhost:8080. Untuk mensimulasikannya, server PHP kedua dijalankan dari folder proyek di port 8080 dengan mode production:
APP_ENV=production php -S localhost:8080 -t public public/index.php
(Pada Windows PowerShell: $env:APP_ENV="production"; php -S localhost:8080 -t public public/index.php.)
Dengan memilih Environment Staging, request yang sama dapat dikirim ke server kedua tanpa mengubah satu pun request di Collection. Perbedaan yang terlihat: alamat server berbeda dan response JSON berupa satu baris karena mode production.



[SCREENSHOT 19: Environment “Staging” dengan base_url baru]



[SCREENSHOT 20: request register atau GET mahasiswa dikirim dengan Environment Staging terpilih, response berhasil]

4.11 Pembahasan

1. Environment memisahkan alamat dari request. Dengan variabel {{base_url}}, perpindahan antara Local dan Staging cukup mengganti Environment, tanpa mengedit puluhan request. Hal ini mengurangi kesalahan dan mempercepat pengujian.
2. Password di-hash, bukan dienkripsi atau disimpan asli. password_hash() menghasilkan hash bcrypt yang memuat salt acak, sehingga password yang sama menghasilkan hash berbeda setiap kali. Jika database bocor, password asli tidak dapat dibaca langsung. Saat login nanti, pencocokan dilakukan dengan password_verify().
3. Hash tidak pernah dikembalikan ke client. Response registrasi hanya berisi id, username, email, dan role, dan ini dicek lewat test otomatis di Postman.
4. Role tidak diambil dari input. Jika role dibaca dari body request, siapa pun dapat mendaftar sebagai admin. Karena itu role selalu memakai default tabel.

5. Pemilihan kode status. 400 dipakai untuk data yang tidak dikirim, 422 untuk data yang dikirim tetapi melanggar aturan (email tidak valid, password terlalu pendek), dan 409 untuk konflik dengan data yang sudah ada (username atau email duplikat).
6. Pengaman berlapis untuk duplikat. Duplikat dicek di Controller agar pesannya jelas, dan constraint UNIQUE di database menjadi pengaman jika ada dua request bersamaan.
7. Batas password 72 karakter. bcrypt hanya memakai 72 byte pertama, sehingga password lebih panjang ditolak agar tidak terpotong secara diam-diam.
8. Register belum memberi akses. Registrasi hanya membuat akun. Proses login yang menghasilkan token (Bearer Token) dan pembatasan akses endpoint ditambahkan pada acara berikutnya.

E. KESIMPULAN

Collection dan dua Environment, yaitu Local dan Staging, berhasil dibuat di Postman dengan menggunakan variabel `{{base_url}}` pada seluruh request tanpa akhiran `.php`. Tabel `users`, Model `User`, `AuthController`, serta pemetaan endpoint `register` pada Router berhasil diimplementasikan untuk melengkapi arsitektur MVC dengan resource non-CRUD pertama. Keamanan kata sandi diterapkan menggunakan fungsi `password_hash()` sehingga kata sandi tersimpan dalam bentuk hash dan tidak pernah dikembalikan kepada client. Endpoint register berhasil menangani kondisi sukses dengan status code 201 serta berbagai kondisi kegagalan dengan status code 400, 405, 409, dan 422, menggunakan format response yang seragam. Hasil pengujian setiap kondisi juga telah didokumentasikan dalam bentuk *Saved Example* pada Collection Postman. Selain itu, tugas mandiri berhasil diselesaikan dengan menerapkan validasi email menggunakan `filter_var()` pada Model `User` dan menyiapkan Environment Staging untuk menyimulasikan penggunaan aplikasi pada lingkungan yang berbeda.

.